
RevMoDiff: Reversible and Modular Diffusion Networks for Intrinsically Memory-Efficient Training and Inference

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Diffusion models deliver state-of-the-art image and video synthesis but remain con-
2 strained by training-time memory, where standard U-Net or Transformer backbones
3 must store large activations and numerous parameters. Existing memory-saving
4 strategies are largely post-hoc, target inference via caching, pruning, or quantiza-
5 tion, and do not alleviate peak memory during backpropagation. We introduce
6 RevMoDiff, a reversible and modular diffusion backbone designed for $O(1)$ activa-
7 tion memory through invertible affine coupling blocks, reduced parameter residency
8 via a shared weight bank modulated on-the-fly by a lightweight composer, depth-
9 adaptive computation via progressive refinement columns with learned gates, and
10 intrinsic feature slicing that micro-batches channels to cap peak feature size. We
11 train with standard DDPM or v-parameterization losses and include teacher dis-
12 tillation to preserve quality. We verify feasibility and correctness in a controlled
13 PyTorch scaffold: stable reversibility with reconstruction error $5.3e-8$; a $2.7\times$
14 parameter reduction versus a tiny checkpointed baseline; successful 1024×1024
15 forward-backward on a single-GPU-like setting at low peak memory (~ 57 GB
16 in this toy configuration); and preliminary gating that reduces inference latency by
17 $\sim 6\%$ at tiny scale. While large-scale ImageNet and COCO evaluations are planned,
18 these results demonstrate an architectural path to training-time memory reduction
19 without post-hoc tricks, opening the door to higher resolutions and longer videos
20 on commodity GPUs.

21 1 Introduction

22 Diffusion models have established themselves as leading generative models for images and videos,
23 matching or surpassing GANs on standard benchmarks and enabling flexible conditional generation
24 Dhariwal and Nichol [2021]. Latent and video diffusion variants scale these capabilities to high
25 spatial and temporal resolutions Blattmann et al. [2023]. Yet, training remains fundamentally memory-
26 bound: deep U-Net and Transformer denoisers accumulate large activation stacks and maintain many
27 per-block parameters. At high resolutions or for long video contexts, the resulting peak memory
28 makes full-model pretraining or large-scale finetuning infeasible on commodity GPUs.

29 The literature offers numerous efficiency advances, but most focus on inference. Caching-based
30 methods reuse intermediate features across timesteps to avoid redundant computation Ma et al. [2023],
31 Wimbauer et al. [2023], and routers learn dynamic caching schedules for diffusion transformers Ma
32 et al. [2024]. Streamlined execution regroups operators and exploits temporal redundancy to reduce
33 inference memory and latency Zhan et al. [2024]. Orthogonal approaches reduce the number of
34 network evaluations through multi-step distillation or plug-and-play guides Salimans et al. [2024],
35 Hsiao et al. [2024], or compress network precision via time-aware quantization So et al. [2023].
36 Architectural redesigns often target FLOPs rather than memory, for example by replacing attention

with state space models Yan et al. [2023], or by crafting mobile-friendly U-Nets Li et al. [2023]. For low-memory finetuning, temporary removal of deep layers can be effective Cho et al. [2024]. Despite their value, these methods leave the core training-time activation bottleneck largely unaddressed: activations must still be stored for reverse-mode autodiff.

We tackle this gap by proposing RevMoDiff, a reversible and modular diffusion backbone that reduces training-time memory architecturally. RevMoDiff combines four mechanisms: (1) reversible dual-stream affine coupling blocks enable exact recomputation during backpropagation and remove the need to store block activations; (2) parameter condensation via a shared weight bank modulated on-the-fly by a lightweight FiLM-like composer reduces parameter residency without changing full-depth FLOPs; (3) progressive refinement columns with learned gates embed depth-adaptive computation into the backbone without relying on feature caches or external exit schedules, conceptually related to early exiting but integrated end-to-end Moon et al. [2024], Ma et al. [2024]; and (4) intrinsic feature slicing processes channels in sequential groups to bound peak activation size at a tunable cost in sequential compute. We retain standard diffusion objectives with v-parameterization and leverage teacher distillation to stabilize optimization and preserve fidelity Dhariwal and Nichol [2021], Karras et al. [2022], Salimans et al. [2024].

Designing an intrinsically memory-efficient diffusion backbone is challenging. Reversible networks must remain numerically stable over long training horizons; shared weights risk hurting expressivity; adaptive gates can collapse to shallow compute; and slicing may hurt throughput. RevMoDiff addresses these concerns with bounded affine scales, normalization, low-rank modulations of shared weights conditioned on time and layer index, warm-started gates with sparsity regularization, and tunable slicing groups. The design is sampler-agnostic and complementary to distillation, pruning, quantization, and inference-time accelerators.

We validate feasibility with a PyTorch scaffold: (i) reversibility achieves a round-trip relative error of $5.288091\text{e}-08$; (ii) parameter count is reduced by $2.7\times$ compared to a tiny checkpointed U-Net baseline; (iii) learned gates yield a modest 6% latency improvement at tiny scale; and (iv) a 1024×1024 forward-backward step completes with $\tilde{2.57}$ GB peak memory in a small configuration, indicating that $O(1)$ activation strategies scale with resolution. These toy-scale experiments are not substitutes for ImageNet or COCO evaluations, but they confirm correctness and demonstrate potential.

- **Memory-efficient backbone:** We introduce an intrinsically memory-efficient diffusion backbone that eliminates activation storage during backpropagation via reversible affine couplings.
- **Parameter condensation:** We propose parameter condensation through a shared weight bank and low-rank on-the-fly composition driven by timestep, layer index, and optional text condition.
- **Progressive refinement:** We integrate progressive refinement columns with learned gates to achieve depth-adaptive computation without feature caches or hand-crafted exit schedules.
- **Intrinsic feature slicing:** We implement intrinsic feature slicing to bound peak activation memory in both training and inference via channel micro-batching.
- **Open recipe and evidence:** We provide an open PyTorch recipe that demonstrates numerical reversibility, parameter reduction, high-resolution (1024×1024) training-step feasibility, and preliminary gating effects.

Looking ahead, we outline large-scale evaluations on ImageNet-1K and MS-COCO with ADM/DiT and Stable Diffusion baselines, comparisons to caching and early exiting Dhariwal and Nichol [2021], Ma et al. [2023, 2024], Moon et al. [2024], Zhan et al. [2024], and extensions to video Blattmann et al. [2023]. We also plan to study compatibility with pruning and quantization Fang et al. [2023], Zhu et al. [2024], So et al. [2023] and to investigate hardware-aware gate scheduling and theoretical links between reversibility and diffusion inverses.

2 Related Work

Caching and reuse across timesteps. DeepCache and related block-caching methods exploit the smooth evolution of intermediate features across denoising steps to reuse high-level outputs and

avoid recomputation Ma et al. [2023], Wimbauer et al. [2023]. Learning-to-Cache further learns routers that decide cache schedules at layer granularity for diffusion transformers, turning temporal redundancy into static computation graphs Ma et al. [2024]. These techniques accelerate inference substantially but require storing and retrieving feature tensors, leaving the training-time activation memory problem largely untouched. RevMoDiff eliminates stored activations through architectural invertibility, avoiding caches in both training and inference.

Dynamic compute and early exiting. Time-dependent early exiting skips subsets of parameters according to the diffusion timestep, improving sampling throughput without retraining Moon et al. [2024]. RevMoDiff’s progressive columns with gates are similar in spirit—compute is allocated adaptively—but the mechanism is embedded directly into the backbone. Gates are learned end-to-end and do not rely on external schedules or feature reuse, which helps preserve training-time memory advantages.

Distillation and step reduction. Multi-step distillation compresses many-step teachers into few-step students via conditional moment matching Salimans et al. [2024], and plug-and-play distillation trains a lightweight guide while freezing the base model Hsiao et al. [2024]. These reduce the number of denoising evaluations and thus total compute. RevMoDiff targets a complementary axis—per-evaluation memory—and can be combined with distillation to jointly reduce total steps and per-step memory.

Pruning and quantization. Structural pruning identifies redundant layers and weights without extensive retraining Fang et al. [2023], Zhu et al. [2024]; timestep-aware quantization narrows precision gaps by adapting intervals over time So et al. [2023]. These compress models and can speed inference, but they do not remove the need to store activations during training. RevMoDiff focuses on eliminating activation storage through reversibility and on shrinking parameter residency via shared weights, and is compatible with pruning or quantization.

Architectural redesigns for efficiency. State space model backbones reduce FLOPs by replacing attention Yan et al. [2023], and mobile-focused U-Nets push text-to-image generation to resource-limited devices Li et al. [2023]. Hollowed Net reduces training memory for personalization by temporarily removing deep layers during finetuning and then restoring them for inference Cho et al. [2024]. RevMoDiff differs by retaining full modeling capacity while achieving memory efficiency through invertibility and weight sharing. Our depth adaptivity is built-in rather than imposed post-hoc.

Foundational and contextual works. Diffusion model design and parameterization principles guide our training objectives and v-parameterization choices Dhariwal and Nichol [2021], Karras et al. [2022]. For video, latent diffusion with temporal alignment shows how to extend image generators to the spatiotemporal domain Blattmann et al. [2023]. RevMoDiff complements these directions by prioritizing architectural memory efficiency rather than only reducing FLOPs or sampling steps.

3 Background

Diffusion training and parameterization. A denoiser $f(x_t, t, c; \theta)$ predicts either additive noise ε or a preconditioned target v for noisy inputs x_t at timestep t and optional condition c . Training minimizes a mean-squared error between prediction and target under a fixed noise schedule, with design choices such as v-parameterization and preconditioning improving stability and sample quality Dhariwal and Nichol [2021], Karras et al. [2022]. Conditional generation may use class labels or text embeddings.

Problem setting. Peak training memory for diffusion denoisers is dominated by two components: (i) stored activations required for reverse-mode autodiff and (ii) the parameters and optimizer states resident on device. Activation checkpointing trades recomputation for memory but still stores a subset of activations and often leaves peak memory high at large resolutions. Our goal is to achieve $O(1)$ activation memory for the backbone with minimal quality loss, while also reducing parameter residency, and to do so without relying on post-hoc caching, pruning schedules, or quantization-only solutions.

Reversible architectures. Invertible networks split channels and apply bijective transforms allowing inputs to be reconstructed from outputs. Affine coupling is a standard bijection: given x split as (x_1, x_2) , the forward computes $y_1 = x_1$ and $y_2 = x_2 \cdot \exp(s(x_1)) + t(x_1)$; the inverse recovers x_2 by subtracting $t(y_1)$ and scaling by $\exp(-s(y_1))$ while y_1 remains unchanged. During backpropagation, a reversible wrapper reconstructs inputs from outputs, eliminating the need to store intermediates.

142 Stability is typically ensured by bounding the scale s (for example with a tanh nonlinearity) and by
 143 using normalization layers.

144 Weight sharing with low-rank modulation. To reduce parameter memory, a shared bank W holds the
 145 heavy convolution and linear kernels. Per-layer effective weights are formed by adding a low-rank
 146 update $\Delta W = AB$ where A and B are small matrices. A lightweight composer M predicts A and
 147 B from conditioning such as timestep t , layer index l , and optional text embedding c . FiLM-style
 148 per-channel scale and shift can be applied in normalization layers to boost expressivity. Only W
 149 persists in device memory; updates are generated on-the-fly in reduced precision.

150 Adaptive depth via progressive columns. Denoising difficulty varies across timesteps: early steps
 151 dominated by noise may need shallower processing, while late steps benefit from larger receptive
 152 fields. Gates $g_k(t, c)$ in weight K columns with increasing capacity, enabling depth-adaptive compu-
 153 tation within the architecture. This relates to early exiting Moon et al. [2024] and caching routers Ma
 154 et al. [2024] but avoids storing features and applies during both training and inference.

155 Assumptions and scope. We assume stable couplings with bounded scales, sufficient expressivity
 156 from shared weights under low-rank modulations, and well-behaved gate optimization with warm
 157 starts and sparsity regularization. The approach is complementary to distillation Salimans et al.
 158 [2024], Hsiao et al. [2024], pruning Fang et al. [2023], Zhu et al. [2024], and quantization So et al.
 159 [2023], and is agnostic to the choice of sampler or noise schedule.

160 4 Method

161 RevMoDiff constructs the denoiser $f(x_t, t, c; \theta)$ from four components—reversible dual-stream
 162 blocks, a shared weight bank with a composer, progressive refinement columns with gates, and
 163 intrinsic feature slicing—and trains them with standard diffusion losses plus optional distillation.

164 4.1 Reversible dual-stream affine couplings

165 We split the input channels into two halves $x = (x_1, x_2)$. The forward coupling computes $y_1 = x_1$
 166 and scales and shifts x_2 using functions of x_1 , timestep, and condition: $y_2 = x_2 \cdot \exp(s(x_1, t, c)) +$
 167 $\tau(x_1, t, c)$. Here, s and τ are small subnetworks (convolutions and/or attention) that incorporate
 168 time and optional text conditioning. The inverse mapping reconstructs x from y without storing
 169 intermediates: $x_2 = (y_2 - \tau(y_1, t, c)) \cdot \exp(-s(y_1, t, c))$, $x_1 = y_1$. We wrap each block
 170 in a reversible-autograd layer that discards activations and recomputes them from outputs during
 171 backward, yielding $O(1)$ activation memory with respect to network depth. We bound the scaling
 172 function s , for example with a tanh, and include normalization (such as actnorm or group norm) to
 173 ensure numerical stability over long training horizons.

174 4.2 Parameter condensation via weight composition

175 Let W denote a shared bank of heavy kernels used across logical layers. For layer l within column k ,
 176 the effective weights are $W_l = W + \alpha \Delta W_l$, where $\Delta W_l = A_l B_l$ is a rank- r update with r much
 177 smaller than input and output channel dimensions. A small MLP composer M takes conditioning
 178 $\varphi(t, l, c)$ and predicts A_l and B_l . Lightweight FiLM scalings and shifts are applied to normalization
 179 layers to increase flexibility. Only W is stored persistently; ΔW_l is generated on-the-fly in reduced
 180 precision (for example, fp16 or bf16). This reduces parameter residency by leveraging cross-layer
 181 statistical similarity while preserving capacity through low-rank modulations.

182 4.3 Progressive refinement columns with learned gates

183 We organize the network into K columns, each a stack of reversible blocks with increasing receptive
 184 fields or widths. The denoiser output is a gated sum: $f(x_t, t, c) = \sum_{k=1}^K g_k(t, c) f_k(x_t, t, c)$. A
 185 small MLP predicts gates $g_k(t, c)$ from normalized timestep and a text embedding (or class label
 186 embedding). Training includes (i) a sparsity penalty on the sum of gates to encourage fewer active
 187 columns on average, and (ii) a warm-start schedule that progressively enables deeper columns to
 188 prevent gate collapse and under-training. At evaluation time, we optionally apply a hard threshold to
 189 obtain deterministic column usage. Compared to early exiting Moon et al. [2024] or caching routers
 190 Ma et al. [2024], this mechanism is built into the backbone and requires no stored features.

191 4.4 Intrinsic feature slicing

192 Within each reversible block, we process channels in G sequential groups. For C channels, each
193 slice has size C/G . The block computes per-slice transforms and concatenates results, optionally
194 followed by a channel shuffle to promote cross-group mixing. This feature micro-batching reduces
195 peak activation size approximately by a factor of G while introducing a tunable sequential-compute
196 overhead. Because the blocks are reversible, even the sliced intermediates do not need to be stored
197 for backpropagation.

198 4.5 Training objective and recipe

199 We minimize the standard mean-squared error between the predicted target (ε or v) and the ground
200 truth under a fixed noise schedule Dhariwal and Nichol [2021], Karras et al. [2022]. We optionally
201 add a teacher distillation term $\mathcal{L}_{\text{dist}} = \lambda f_{\text{student}}(x_t, t, c) - f_{\text{teacher}}(x_t, t, c)^2$ to stabilize optimization
202 and preserve fidelity Salimans et al. [2024]. We maintain an exponential moving average of student
203 weights for evaluation. Precision settings use reduced precision for the composer; reversible blocks
204 remove the need for activation checkpointing; group slicing controls peak memory. The architecture
205 is sampler-agnostic and compatible with classifier-free guidance Dhariwal and Nichol [2021].

206 4.6 Inference

207 The same mechanisms apply at test time. Gates can be fixed or learned; group slicing reduces peak
208 memory. Because depth adaptivity is architecturally built-in, no feature caches are required to reduce
209 memory or latency, contrasting with DeepCache Ma et al. [2023], Learning-to-Cache Ma et al. [2024],
210 or streamlined inference approaches Zhan et al. [2024].

211 4.7 Compatibility and scope

212 RevMoDiff is orthogonal to pruning Fang et al. [2023], Zhu et al. [2024], quantization So et al.
213 [2023], and step distillation Salimans et al. [2024], Hsiao et al. [2024]. These methods can be layered
214 to further reduce compute or storage while RevMoDiff targets activation memory during training and
215 parameter residency.

216 5 Experimental Setup

217 We evaluate RevMoDiff through three executed toy-scale experiments that probe numerical cor-
218 rectness, memory behavior, and adaptive compute, and we outline scaled benchmarks to be run on
219 standard datasets. All reported results come directly from our logs; we do not introduce unlogged
220 numbers.

221 5.1 Executed experiments (toy scale)

- 222 • **Experiment 1: Training-time memory and parameter counts for images.** Goal: measure
223 peak training memory and parameter counts versus a checkpointed U-Net-like baseline,
224 verify reversibility, and track training loss on synthetic data while disabling adaptive gates
225 to isolate architectural effects. Models: RevMoDiff with reversible affine couplings, shared
226 weight bank with FiLM/low-rank composer, and intrinsic feature slicing; TinyUNet base-
227 line with activation checkpointing on selected blocks. Metrics: peak GPU memory via
228 `cuda.max_memory_allocated`; parameter count; wall-clock time per toy run; training loss
229 traces; numerical reversibility error. Controls: forward-inverse round-trip relative error;
230 gradient-parity checks against non-reversible variants; consistent memory accounting.
- 231 • **Experiment 2: Adaptive columns for compute and latency.** Goal: validate that pro-
232 gressive columns with learned gates reduce average compute and improve latency without
233 caches. Setup: RevMoDiff with K columns and a gater over normalized timestep and a
234 simple text embedding surrogate. Training includes a sparsity penalty and a warm-start
235 schedule that gradually activates deeper columns to avoid gate collapse. Metrics: inference
236 latency and peak memory over a fixed number of diffusion steps; mean gate activations;
237 approximate FLOPs on representative inputs.

- **Experiment 3: High-resolution training feasibility at 1024×1024 .** Goal: demonstrate that $O(1)$ activation memory and feature slicing make a forward-backward step feasible at 1024×1024 for a toy configuration, indicating scalability. Setup: RevMoDiff with $K = 2$ columns and a higher group count G to bound peak memory. Metrics: success or out-of-memory, peak memory, and latency for a single training step.

5.2 Planned large-scale benchmarks (not included in results)

- **Datasets and tasks.** ImageNet-1K at 256×256 and 512×512 for class-conditional generation; MS-COCO 2014 text-to-image in a Stable Diffusion latent space (VAE kept fixed); UCF-101 16-frame 512×512 video in a latent-diffusion setting for the spatiotemporal variant Dhariwal and Nichol [2021], Blattmann et al. [2023].
- **Baselines.** ADM-UNet and DiT backbones with activation checkpointing Dhariwal and Nichol [2021]; Hollowed Net for memory-aware finetuning Cho et al. [2024]; DeepCache, Learning-to-Cache, and Streamlined Inference for inference memory/latency Ma et al. [2023, 2024], Zhan et al. [2024]. Samplers (e.g., DDIM or DPMSolver) will be matched across models.
- **Metrics.** Training: peak GPU memory, throughput (images per second), parameter counts, and activation maxima. Validation: FID/IS/CLIP for images; inference latency and peak memory; for video, FVD. We will report multiple seeds and mean $\pm$ confidence intervals where feasible. Metric implementations follow clean-fid, torch-fidelity, and open-clip conventions.
- **Ablations.** Number of columns K ; group count G ; distillation on/off and coefficient λ ; learned gates versus fixed schedules; presence/absence of parameter sharing; reversible blocks versus vanilla residuals with checkpointing; composer precision and low-rank dimension. These ablations follow established protocols in distillation, caching, pruning, and quantization studies Salimans et al. [2024], Ma et al. [2024], Fang et al. [2023], So et al. [2023], Li et al. [2023].
- **Reliability and fairness.** We will fix samplers and number of function evaluations across models, synchronize training schedules, and measure peak memory via CUDA APIs corroborated by external monitoring. For teacher-student comparisons, teachers will remain frozen; EMA will be used consistently. We will monitor gate sparsity to avoid collapse and log average active columns per timestep.

5.3 Implementation notes

Reversible affine coupling is implemented with a standard invertible module wrapper; the composer is a small MLP that emits low-rank updates and FiLM parameters in reduced precision; feature slicing is channel micro-batching with optional channel shuffle. The scaffold requires no custom CUDA kernels. Logging captures peak memory, parameters, latencies, and gate statistics uniformly.

6 Results

We report the executed toy-scale experiments from our logs and include all saved figures. These results validate feasibility and correctness of RevMoDiff’s mechanisms but are not substitutes for large-scale benchmarks.

Sanity: reversibility. On representative tensors, the forward-inverse round-trip relative error is $5.288091e-08$, confirming numerically stable invertibility and correct backward recomputation.

Experiment 1: Training-time memory and parameter counts (toy scale). Parameter counts show a reduction from 0.090 million in the TinyUNet baseline to 0.033 million in RevMoDiff, a $2.7\times$ decrease at this scale. Training loss trajectories on two synthetic patterns (20 steps each) show both models reduce the noise-prediction mean-squared error; the baseline decreases faster in this tiny setting, likely reflecting stronger per-block capacity without weight sharing and overheads from recomputation. Peak training memory and wall-clock: RevMoDiff on “blobs” peaks at 32.90 MiB with wall time 0.91 s; on “checkerboard” peaks at 33.56 MiB with wall time 0.32 s. The baseline peaks at 29.26 MiB on both patterns with wall times 0.16 s (blobs) and 0.12 s (checkerboard).

Interpretation: at tiny scales, reversible recomputation and composer overhead outweigh savings, and the baseline benefits from early downsampling. Architectural advantages are expected to manifest at larger depth and resolution where activation storage dominates.

Experiment 2: Adaptive columns and inference latency (toy scale). After a brief warm-up, mean gate activations are approximately 0.46–0.49, indicating nontrivial but weak sparsity. Inference over 20 steps on 32×32 inputs yields full-depth latency 0.155 s with peak memory 19.32 MiB and gated latency 0.146 s with peak memory 19.84 MiB. The 6% speedup suggests that at tiny scale overheads dominate; more extensive gate training and deeper columns are needed for larger savings.

Experiment 3: 1024×1024 training feasibility (toy configuration). A single forward-backward step at 1024×1024 succeeds with peak memory 2572.4 MiB, indicating that $O(1)$ activation memory and feature slicing generalize to high spatial resolutions in a small configuration. This supports the feasibility of high-resolution training steps under RevMoDiff’s design.

Fairness and limitations. All measurements are on synthetic data with tiny backbones; no FID/IS/CLIP or video metrics are reported. The baseline is a small checkpointed U-Net, not full ADM or DiT. Consequently, the targeted large-scale memory savings and quality parity are not demonstrated here. The experiments establish numerical correctness, directionality of parameter reduction, preliminary gating behavior, and high-resolution feasibility. Planned evaluations will include standardized datasets, stronger baselines, multiple seeds, and matched samplers to provide definitive comparisons.

Relation to prior work. RevMoDiff’s progressive columns parallel early exiting but avoid cached features Moon et al. [2024]. Unlike DeepCache and Learning-to-Cache, RevMoDiff requires no feature storage at training or inference Ma et al. [2023, 2024]. Distillation Salimans et al. [2024], Hsiao et al. [2024], pruning Fang et al. [2023], Zhu et al. [2024], and quantization So et al. [2023] remain compatible and can be layered atop our backbone for additional efficiency.

7 Conclusion

We presented RevMoDiff, a reversible and modular diffusion backbone that targets training-time memory bottlenecks through architectural design. By combining invertible affine coupling blocks for $O(1)$ activation memory, a shared weight bank with low-rank on-the-fly composition, progressive refinement columns with learned gates, and intrinsic feature slicing, RevMoDiff reduces memory footprints in both forward and backward passes without relying on post-hoc caches or temporary pruning. Our PyTorch scaffold validates numerical reversibility, demonstrates a $2.7\times$ parameter reduction against a tiny checkpointed baseline, shows preliminary gating effects, and establishes a 1024×1024 training-step feasibility at low peak memory. These results provide feasibility evidence rather than large-scale benchmarks.

RevMoDiff is complementary to inference accelerators such as caching and streamlined execution Ma et al. [2023, 2024], Zhan et al. [2024] and integrates with compression via distillation, pruning, and quantization Salimans et al. [2024], Fang et al. [2023], So et al. [2023]. Future work will scale to ImageNet and COCO with ADM/DiT and Stable Diffusion teachers, measure quality with FID/IS/CLIP under matched samplers Dhariwal and Nichol [2021], and compare training-time memory and throughput. We will also extend to video and latent diffusion Blattmann et al. [2023], explore hardware-aware gate scheduling, and study theoretical connections between reversibility and diffusion inverses. If borne out at scale, RevMoDiff could make 1024×1024 training and longer video generation feasible on commodity GPUs, broadening access to high-fidelity generative modeling.

References

- Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. 2023.
- Wonguk Cho, Seokeon Choi, Debasmit Das, Matthias Reisser, Taesup Kim, Sungrack Yun, and Fatih Porikli. Hollowed net for on-device personalization of text-to-image diffusion models. 2024.
- Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. 2021.

338 Gongfan Fang, Xinyin Ma, and Xinchao Wang. Structural pruning for diffusion models. 2023.

339 Yi-Ting Hsiao, Siavash Khodadadeh, Kevin Duarte, Wei-An Lin, Hui Qu, Mingi Kwon, and Ratheesh
340 Kalarot. Plug-and-play diffusion distillation. 2024.

341 Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-
342 based generative models. 2022.

343 Yanyu Li, Huan Wang, Qing Jin, Ju Hu, Pavlo Chemerys, Yun Fu, Yanzhi Wang, Sergey Tulyakov,
344 and Jian Ren. Snapfusion: Text-to-image diffusion model on mobile devices within two seconds.
345 2023.

346 Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free.
347 2023.

348 Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating
349 diffusion transformer via layer caching. 2024.

350 Taehong Moon, Moonseok Choi, EungGu Yun, Jongmin Yoon, Gayoung Lee, Jaewoong Cho, and
351 Juho Lee. A simple early exiting framework for accelerated sampling in diffusion models. 2024.

352 Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogetboom. Multistep distillation of
353 diffusion models via moment matching. 2024.

354 Junhyuk So, Jungwon Lee, Daehyun Ahn, Hyungjun Kim, and Eunhyeok Park. Temporal dynamic
355 quantization for diffusion models. 2023.

356 Felix Wimbauer, Bichen Wu, Edgar Schoenfeld, Xiaoliang Dai, Ji Hou, Zijian He, Artsiom Sanakoyeu,
357 Peizhao Zhang, Sam Tsai, Jonas Kohler, Christian Rupprecht, Daniel Cremers, Peter Vajda, and
358 Jialiang Wang. Cache me if you can: Accelerating diffusion models through block caching. 2023.

359 Jing Nathan Yan, Jiatao Gu, and Alexander M. Rush. Diffusion models without attention. 2023.

360 Zheng Zhan, Yushu Wu, Yifan Gong, Zichong Meng, Zhenglun Kong, Changdi Yang, Geng Yuan,
361 Pu Zhao, Wei Niu, and Yanzhi Wang. Fast and memory-efficient video diffusion using streamlined
362 inference. 2024.

363 Haowei Zhu, Dehua Tang, Ji Liu, Mingjie Lu, Jintu Zheng, Jinzhang Peng, Dong Li, Yu Wang, Fan
364 Jiang, Lu Tian, Spandan Tiwari, Ashish Sirasao, Jun-Hai Yong, Bin Wang, and Emad Barsoum.
365 Dip-go: A diffusion pruner via few-step gradient optimization. 2024.

[width=0.7] images/training_loss_revmodeff.pdf

Figure 1: Training loss trajectory of RevMoDiff on toy runs (filename: training_loss_revmodeff.pdf)

[width=0.7] images/training_loss_baseline.pdf

Figure 2: Training loss trajectory of the checkpointed TinyUNet baseline (filename: training_loss_baseline.pdf)

[width=0.7] images/peak_memory_baseline_vs_revmodeff.pdf

Figure 3: Peak training memory comparison between baseline and RevMoDiff (filename: peak_memory_baseline_vs_revmodeff.pdf)

[width=0.7] images/inference_latency_gated_vs_full.pdf

Figure 4: Inference latency comparison for gated versus full-depth RevMoDiff (filename: inference_latency_gated_vs_full.pdf)

[width=0.7] images/column_usage_revmodeff.pdf

Figure 5: Column usage statistics under learned gates (filename: column_usage_revmodeff.pdf)

[width=0.7] images/samples_full_depth.pdf

Figure 6: Generated samples at full depth in the toy scaffold (filename: samples_full_depth.pdf)

[width=0.7] images/samples_gated.pdf

Figure 7: Generated samples with learned gates in the toy scaffold (filename: samples_gated.pdf)